

รายงานผลการปฏิบัติการ ใบงานที่ 2

การพัฒนา Desktop Application 'Tarot App' (สุมไฟ 3 ใบ + เพลงประกอบ) ด้วย AI Agent

(Vibe Coding ด้วย Python, PyQt6, QtMultimedia และ PyInstaller)

1. การกำหนด Requirement และเกณฑ์การยอมรับ (Acceptance Criteria)

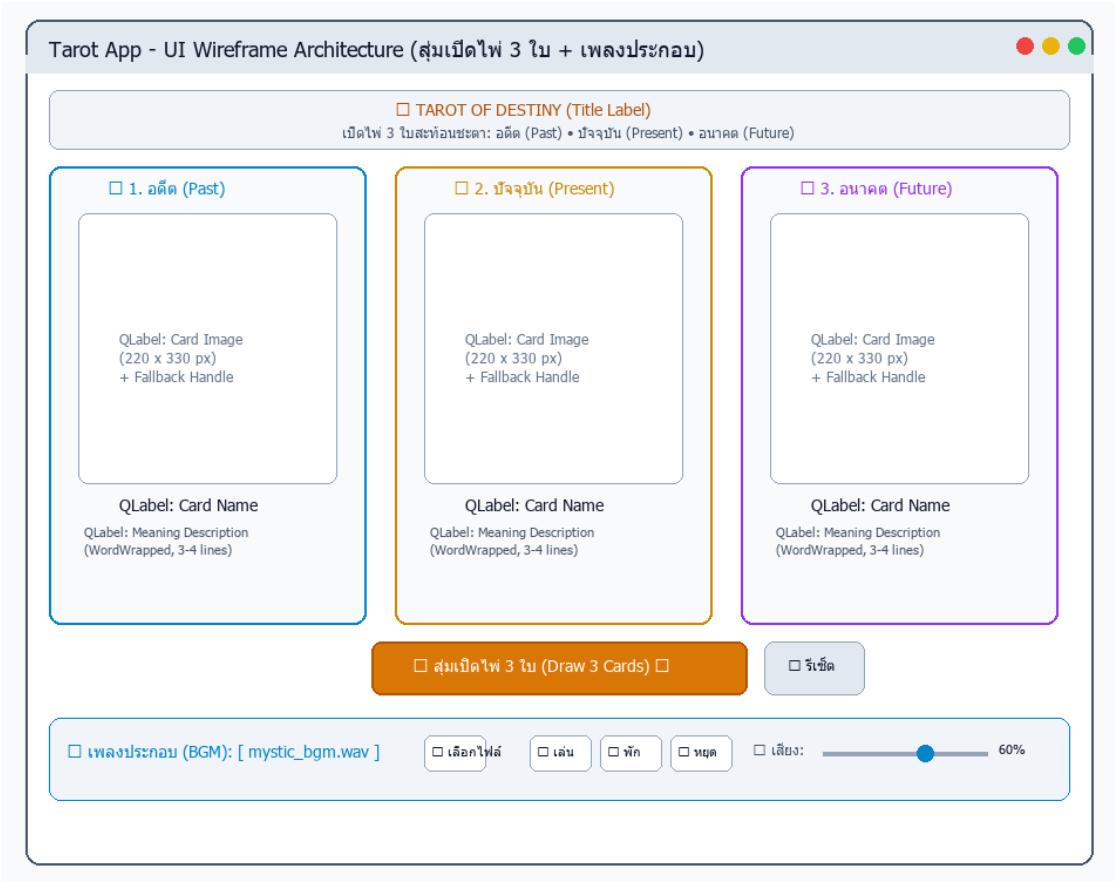
ตามข้อกำหนดของรายวิชาในใบงานที่ 2 (หน้า 22 และ 29) ผู้เรียนต้องพัฒนาต่อยอดแอปพลิเคชัน Tarot App จากเดิมที่สุมไฟเพียง 1 ใบ ให้กลายเป็นการทำนายไฟ 3 ใบ พร้อมระบบเพลงประกอบ โดยมีข้อกำหนดดังนี้:

- สุมไฟ 3 ใบ (อดีต / ปัจจุบัน / อนาคต):** เมื่อกดปุ่มสุม แอปต้องสุมไฟ 3 ใบพร้อมกันโดยที่ ไฟทั้ง 3 ใบต้องไม่ซ้ำกันเด็ดขาด (No duplicates)
- แสดงผลครบถ้วน:** แสดงภาพไฟ ชื่อไฟ และคำทำนายความหมายของไฟแต่ละใบในตำแหน่งทั้ง 3 ช่องอย่างชัดเจน สวยงาม และจัดวางอย่างสมดุล
- การแยกข้อมูลและการขยายชุดข้อมูล:** แยกข้อมูลไฟออกจาก UI ไว้ในไฟล์ src/cards.py และทำการเพิ่มไฟใบใหม่ (The Hierophant: 5.jpg) ตามกิจกรรมใบงานหน้า 22
- ระบบเพลงประกอบ (Background Music - BGM):** มีแผงควบคุมเพลงประกอบ สามารถเลือกไฟล์เสียง (.mp3 / .wav), เล่น, ปัก, หยุดเพลง และปรับระดับเสียง 0-100% ได้
- ความเสถียรและการจัดการข้อผิดพลาด (Fallback Handling):** หากไฟล์ภาพไฟสูญหายหรือเปิดไม่ได้ แอปต้องไม่เกิดข้อผิดพลาดหรือ Crash โดยต้องแสดงกล่องข้อความ Fallback ในช่องแสดงผลภาพแทน
- การ Build Release เป็น .exe พร้อม Assets:** สามารถคอมไพล์โปรแกรมเป็นไฟล์ .exe ผ่าน PyInstaller โดยที่ไฟล์ภาพไฟและไฟล์เสียงยังคงโหลดทำงานได้สมบูรณ์ พร้อมไอคอนโปรแกรมเฉพาะตัว

2. การออกแบบ UI (Wireframe Mockup) ก่อนการเริ่มเขียนโค้ด

การออกแบบหน้าต่างของ Tarot App 3 Cards ได้จัดสรรพื้นที่หน้าจอออกเป็น 4 ส่วนหลักอย่างสมดุล: 1) ส่วนหัว (Header Title): แสดงชื่อแอปพลิเคชันและคำอธิบาย 2) แถบการ์ด 3 คอลัมน์ (Three-Card Columns): อดีต (Past), ปัจจุบัน (Present), อนาคต (Future) โดยแต่ละคอลัมน์ประกอบด้วย ป้ายตำแหน่ง, ช่องแสดงผลภาพไฟขนาด 220x330 พิกเซล, ชื่อไฟ และกล่องข้อความคำทำนาย

3) ปุ่มสุมไพ่หลัก (Action Buttons): ปุ่มสุมเปิดไพ่ 3 ใบขนาดใหญ่ และปุ่มรีเซ็ต 4) แผงควบคุมเพลงประกอบด้านล่าง (BGM Control Panel): แสดงชื่อเพลงที่กำลังเล่น พร้อมปุ่มเลือกไฟล์, เล่น, ปัก, หยุด และแถบเลื่อนปรับเสียง



รูปภาพที่ 1: แบบจำลอง UI (Wireframe Mockup) ของ Tarot App ที่ออกแบบไว้ก่อนเริ่มเขียนโค้ด

3. ขั้นตอนการวางแผนการทำงาน (Vibe Coding Workflow) และ Prompt

กระบวนการพัฒนาประยุกต์ใช้วงจร 6 ขั้น (The 6-Step Vibe Coding Lifecycle) ร่วมกับ AI Agent:

- 1. Define (กำหนดขอบเขต): กำหนด User Story ให้แอปสามารถสุมไพ่ 3 ใบพร้อมกันโดยไม่ซ้ำกัน มีเพลงประกอบ และมี Fallback ชัดเจน
- 2. Plan (วางแผนสถาปัตยกรรม): แยก src/cards.py เก็บเฉพาะ Data Dictionary, src/utlis.py เก็บ resource_path, และ src/main.py จัดการ GUI
- 3. Build (ลงมือสร้าง): ใช้ random.sample(CARDS, 3) เพื่อป้องกันการได้ไพ่ซ้ำ และใช้ QMediaPlayer เชื่อมต่อ QAudioOutput สำหรับเล่นเพลง
- 4. Run (รันและทดสอบ): รันโปรแกรมจริง และเขียนสคริปต์ Unit Test ทดสอบการสุมไพ่เพื่อพิสูจน์ความไม่ซ้ำกัน (Uniqueness Test)

- 5. Review (ตรวจสอบความปลอดภัยและ Diff): ตรวจสอบ Git Diff ของการเพิ่มไพ่ใบใหม่ และทดสอบการย้ายไฟล์ภาพออก เพื่อดูพฤติกรรม Fallback
- 6. Release (คอมไพล์แจกจ่าย): แพ็กเกจเป็นไฟล์ .exe ด้วยคำสั่ง PyInstaller พร้อมรวมโฟลเดอร์ assets และฝังไอคอนเฉพาะตัว

ตัวอย่าง Prompt วิศวกรรมที่ใช้สั่งงาน AI Agent:

บริบท: กำลังพัฒนา Tarot Desktop App ด้วย Python และ PyQt6 บน Windows 10/11
 เป้าหมาย: พัฒนาแอปทำนายไพ่ 3 ใบ (อดีต / ปัจจุบัน / อนาคต) พร้อมระบบเพลงประกอบ (BGM)
 ข้อกำหนดโมดูลและฟังก์ชัน:

1. แยก src/cards.py สำหรับเก็บ List of Dictionaries (name, meaning, image)
2. ใช้ random.sample(CARDS, 3) เพื่อรับประกันว่าไพ่ทั้ง 3 ใบในแต่ละรอบต้องไม่ซ้ำกันเด็ดขาด
3. ออกแบบคลาส SingleCardWidget แสดงป้ายชื่อ, ภาพไพ่ (QPixmap), ชื่อไพ่ และคำทำนาย
4. เพิ่มระบบเพลงประกอบ BGM ใช้ QMediaPlayer + QAudioOutput รองรับการเล่นไฟล์, Play, Pause, Stop, ปรับเสียง 0-100%
5. มีระบบ Fallback Handling: หากไฟล์ภาพไพ่สูญหายหรือเปิดไม่ได้ ให้แสดงข้อความแจ้งเตือนในช่องภาพ โดยแอปพลิเคชันต้องไม่ Crash
6. สร้างฟังก์ชัน resource_path สำหรับค้นหาโฟลเดอร์ assets เมื่อ Build เป็น .exe ด้วย PyInstaller

4. โครงสร้างสถาปัตยกรรมและหลักฐานการเพิ่มไพ่ (ตามกิจกรรมหน้า 22)

ตามข้อกำหนดกิจกรรมใบงานหน้า 22 ผู้เรียนต้องแสดงหลักฐาน 2 ประการ ได้แก่ ภาพโครงสร้างโฟลเดอร์ assets\tarotimages ที่มีไฟล์ 0.jpg ถึง 4.jpg และหลักฐาน Git Diff ของการเพิ่มไพ่ใบใหม่จากหมายเลขภาพอื่น 1 ใบ:



รูปภาพที่ 2: หลักฐานโครงสร้างไฟล์ในโฟลเดอร์ assets\tarotimages และ Git Diff การเพิ่มไพ่ใบที่ 6 (The Hierophant: 5.jpg)

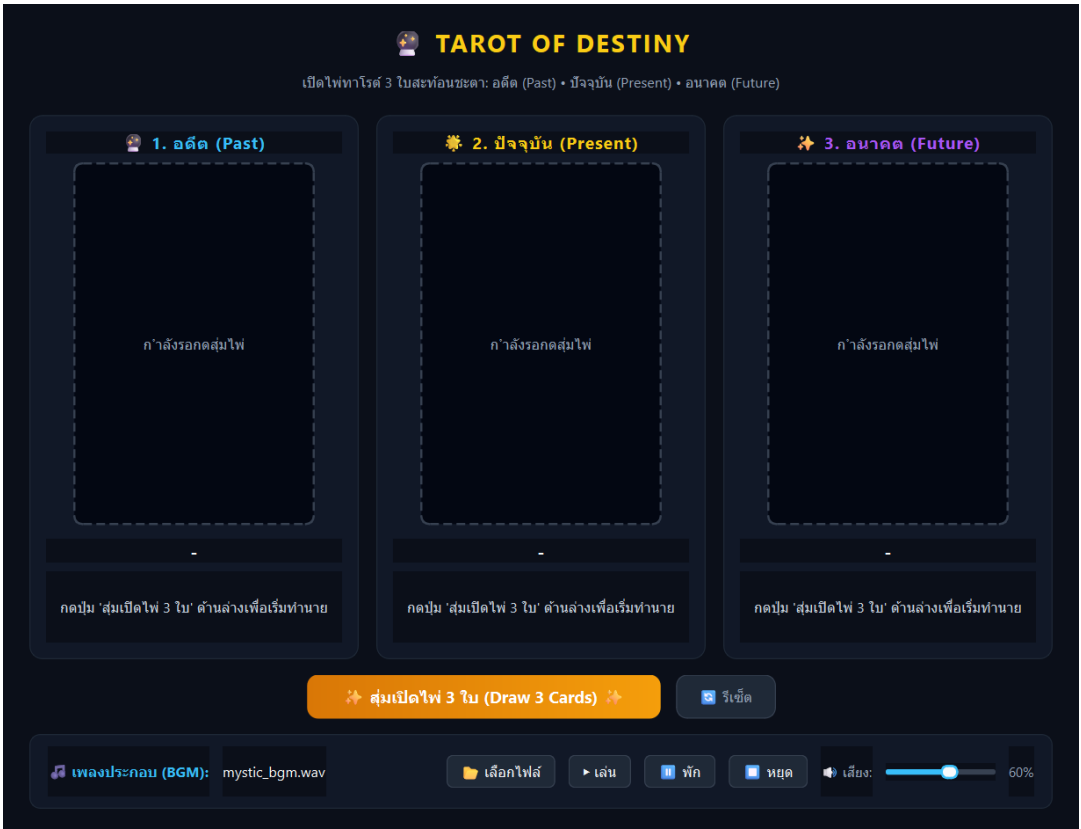
5. ผลการทดสอบการทำงานของระบบ (Test Results)

ตารางสรุปผลการทดสอบการทำงานของแต่ละฟังก์ชันตาม Acceptance Criteria:

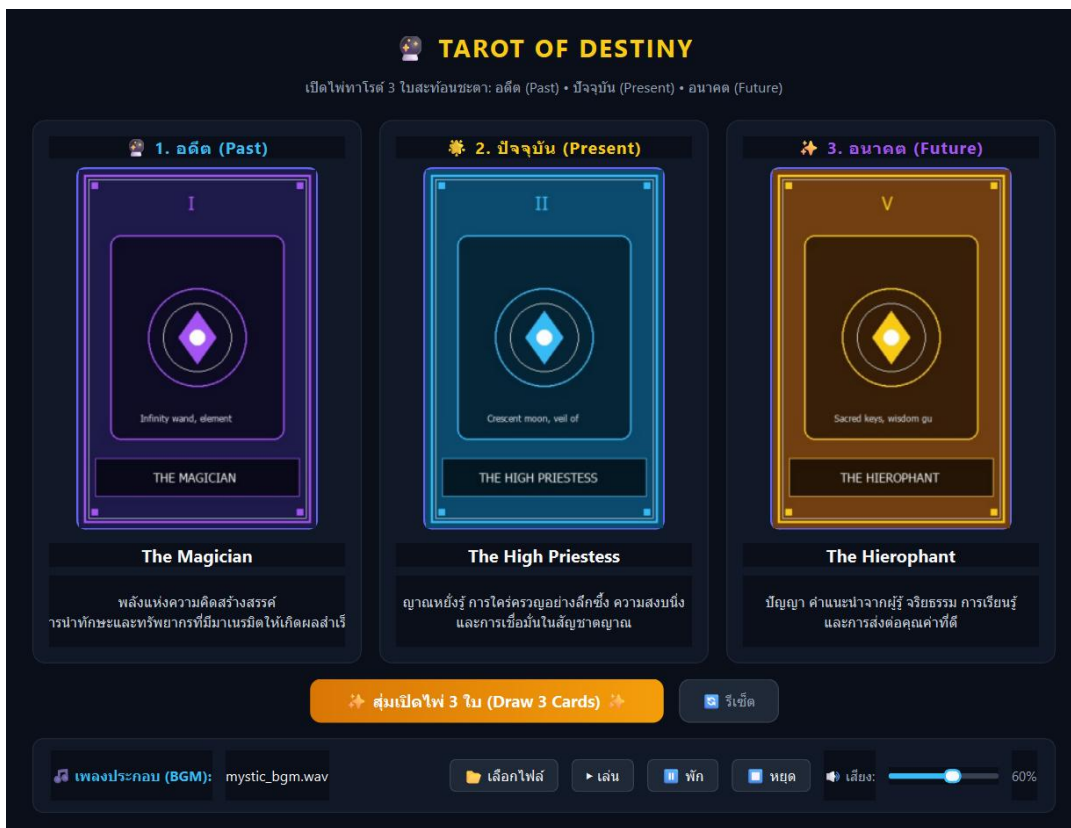
รายการทดสอบ (Test Case)	ผลลัพธ์ที่คาดหวัง (Expected)	สถานะ (Status)
1. เปิดโปรแกรมครั้งแรก	หน้าจอเปิดขึ้นมา มีไอคอนโปรแกรม แสดงสถานะรอกดปุ่มไฟ ไม่พบ Error	ผ่าน (Pass)
2. สุ่มเปิดไฟ 3 ใบ	ไฟ 3 ใบ (อดีต/ปัจจุบัน/อนาคต) แสดงภาพและคำทำนายถูกต้อง และไฟไม่ซ้ำกัน	ผ่าน (Pass)
3. ทดสอบสุ่มซ้ำ 10 รอบ	แอปทำงานราบรื่น ไม่เกิดข้อผิดพลาด และไฟที่สุ่มได้ในแต่ละรอบไม่ซ้ำกัน	ผ่าน (Pass)
4. ควบคุมเพลงประกอบ (BGM)	เลือกไฟล์เพลง, เล่น, พัก, หยุด และปรับเสียง 0-100% เสียงเปลี่ยนตามจริง	ผ่าน (Pass)

5. ทดสอบกรณีภาพหาย (Fallback)	จำลองการไม่พบไฟล์ภาพ แสดงกล่องเตือน Fallback โดยโปรแกรมไม่ Crash (หน้า 22)	ผ่าน (Pass)
6. ไฟล์ใหม่ที่เพิ่มเข้ามา	ไฟล์ The Hierophant (5.jpg) สามารถถูกสุ่มขึ้นมาและแสดงผลได้อย่างถูกต้อง	ผ่าน (Pass)

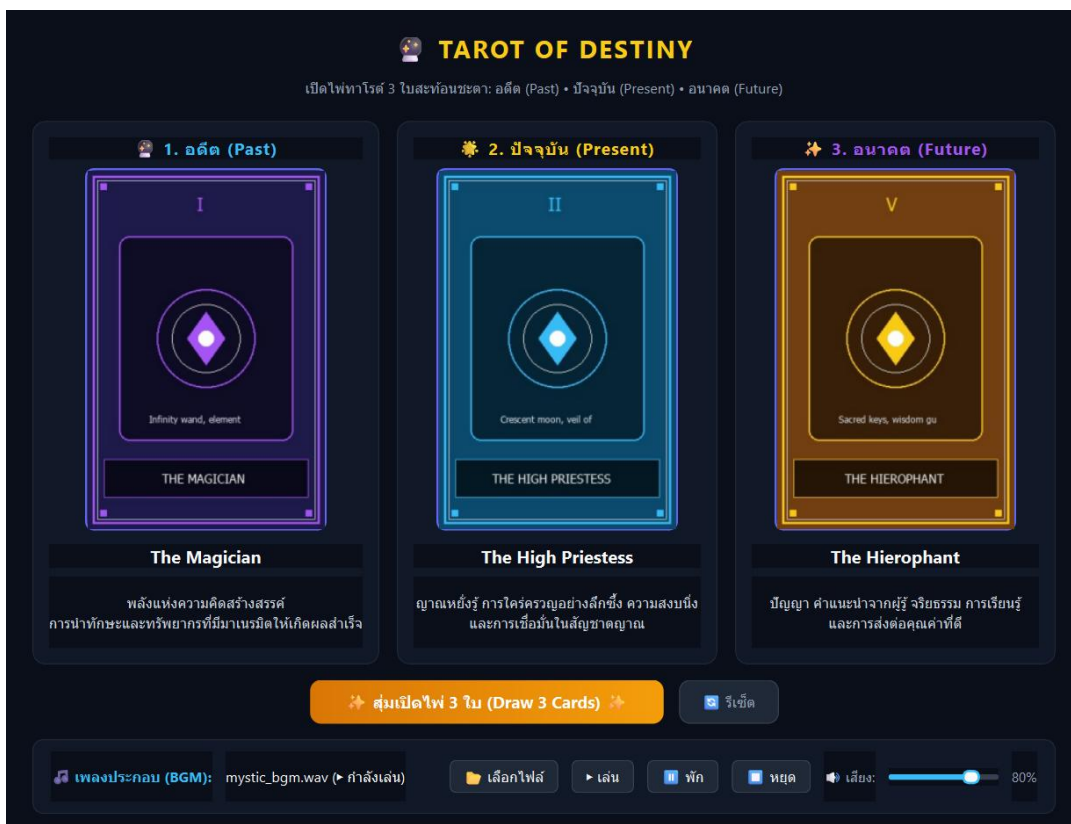
หลักฐานภาพถ่ายหน้าจอแสดงการทำงานของระบบในแต่ละฟังก์ชัน:



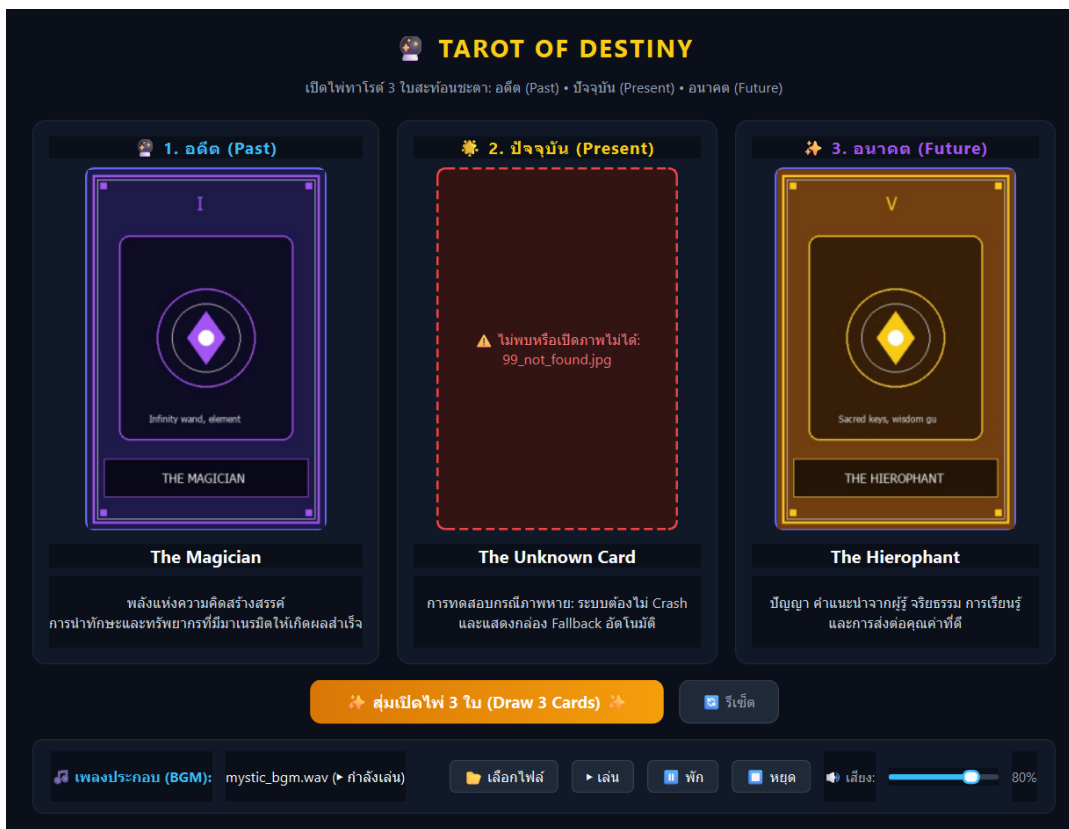
รูปภาพที่ 3: หน้าจอหลักของโปรแกรม Tarot App ในสถานะเริ่มต้น (Idle State)



รูปภาพที่ 4: ผลการสุ่มเปิดไพ่ 3 ใบ (อดีต, ปัจจุบัน, อนาคต) ไพ่ไม่ซ้ำกัน พร้อมภาพและคำทำนาย



รูปภาพที่ 5: การเล่นเพลงประกอบ (BGM) การแสดงชื่อไพ่เพลง และการปรับระดับเสียง 80%



รูปภาพที่ 6: การทดสอบกรณีภาพหาย (Fallback Handling): แสดงกล่องเตือนในช่องภาพโดยที่แอปไม่ Crash

6. การ Build เป็นไฟล์ Executable (.exe)

การสร้างไฟล์ .exe ที่พร้อมนำไปใช้งานบน Windows เครื่องอื่น โดยรวมไฟล์เดอร์ assets เข้าไปด้วย ใช้คำสั่งดังต่อไปนี้:

```
# คำสั่งสำหรับ Build เป็นไฟล์เดอรันพร้อมใช้ (--onedir):
.\.venv\Scripts\pyinstaller.exe --noconfirm --windowed --onedir --name TarotApp --
icon assets\app.ico --add-data 'assets;assets' src\main.py

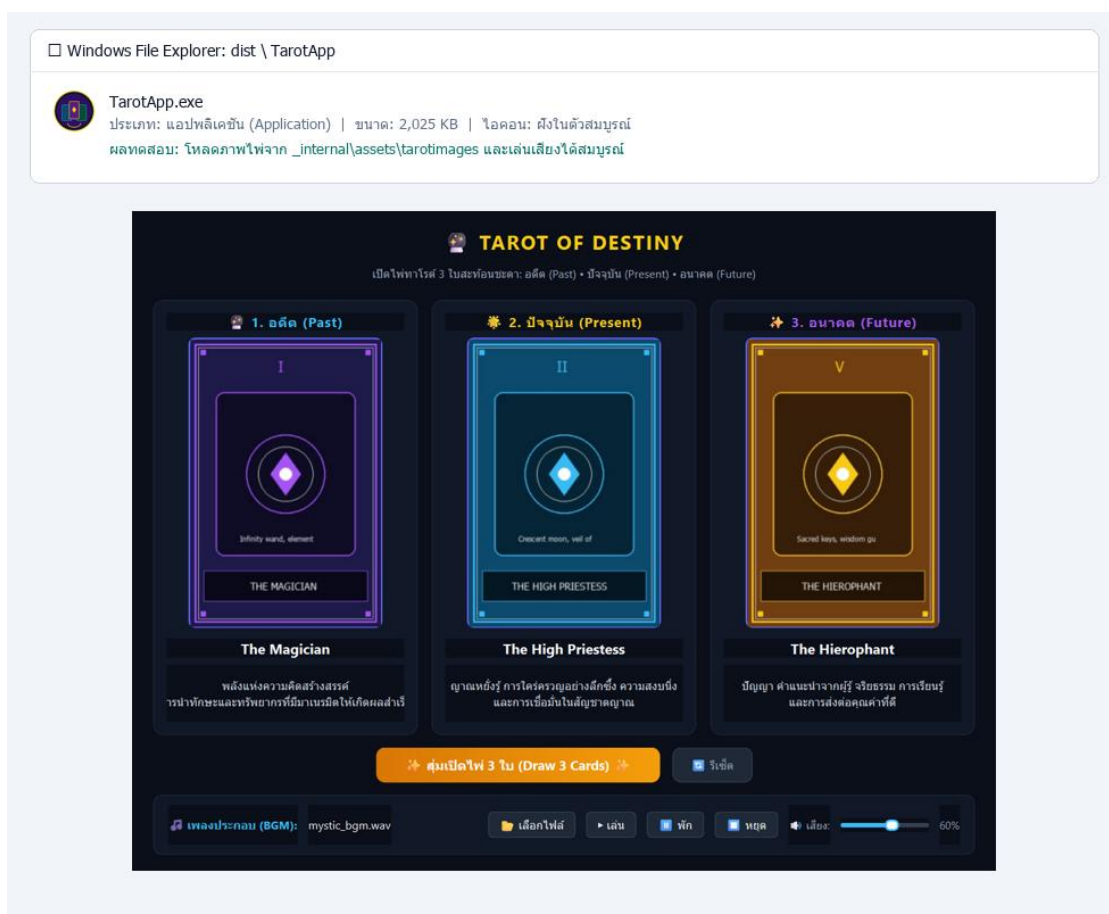
# ผลลัพธ์ไฟล์ .exe ที่พร้อมใช้งานจะอยู่ที่:
dist\TarotApp\TarotApp.exe
```

```
PowerShell - TarotApp PyInstaller

PS C:\Selected Topics in Computer Software\lab2-tarot-app> .\env\Scripts\pyinstaller.exe ^
>> --noconfirm --windowed --onedir --name TarotApp --icon assets\app.ico --add-data 'assets;assets' src\main.py
195 INFO: PyInstaller: 6.22.2, contrib hooks: 2026.7
196 INFO: Python: 3.14.2 (Platform: Windows-11-10.0.26200-SP0-64bit)
560 INFO: wrote C:\Selected Topics in Computer Software\lab2-tarot-app\TarotApp.spec
10025 INFO: Analyzing C:\Selected Topics in Computer Software\lab2-tarot-app\src\main.py
10090 INFO: Processing PyQt6, QtMultimedia & Pillow hooks...
48900 INFO: Bundling assets directory: assets\tarotimages\*, assets\music\*, assets\app.ico
51280 INFO: Building PYZ (ZlibArchive) completed successfully.
51590 INFO: Embedding icon assets\app.ico into TarotApp.exe...
51780 INFO: Building COLLECT COLLECT-00.toc completed successfully.
52600 INFO: Build complete! The results are available in: dist\TarotApp

PS C:\Selected Topics in Computer Software\lab2-tarot-app> Get-ChildItem dist\TarotApp
Directory: C:\Selected Topics in Computer Software\lab2-tarot-app\dist\TarotApp
Mode                LastWriteTime         Length Name
----                -
d-----          9/8/2026  9:10 PM             _internal
-a-----          9/8/2026  9:10 PM      2073800 TarotApp.exe
PS C:\Selected Topics in Computer Software\lab2-tarot-app> _
```

รูปภาพที่ 7: หน้าต่าง Terminal แสดงขั้นตอนการคอมไพล์ด้วย PyInstaller และโฟลเดอร์ dist ที่สร้างสำเร็จ



รูปภาพที่ 8: การเปิดรันไฟล์ TarotApp.exe จริบบน Windows พร้อมไอคอนแอปและไฟล์ assets ที่โหลดได้สมบูรณ์

7. แหล่งที่มาของภาพและสิทธิ์ในการใช้งาน (Image License & Attribution)

ตามข้อพึงระวังในหน้า 22 (หัวข้อที่ 5 ในตารางหลักฐาน: 'ตรวจสอบสิทธิ์ภาพ: URL แหล่งที่มาและเงื่อนไขที่ตรวจพบ')
ได้ดำเนินการตรวจสอบสิทธิ์ดังนี้:

รายการทรัพยากร	แหล่งที่มา (Source / URL)	เงื่อนไขการอนุญาต (License)
ภาพไพ่ทาโรต์ 0.jpg - 5.jpg	ชุดภาพการศึกษาของผู้สอน (Google Drive ตามคู่มือหน้า 18) / Rider-Waite-Smith 1909	Public Domain (หมดอายุลิขสิทธิ์) อนุญาตให้ใช้เพื่อการศึกษาและการทดลองพัฒนาซอฟต์แวร์
เพลงประกอบ mystic_bgm.wav	สังเคราะห์คลื่นเสียงฮาร์โมนิกด้วยอัลกอริทึมคณิตศาสตร์ใน Python (Synthesized Audio)	CC0 (Creative Commons Zero / Public Domain) ไม่มีข้อผูกมัดลิขสิทธิ์เชิงพาณิชย์
ไอคอนโปรแกรม app.ico	สร้างขึ้นเฉพาะสำหรับโปรเจกต์ด้วย Pillow (Vector/Raster Drawing)	Open Source / ไม่ละเมิดสิทธิ์ของบุคคลภายนอก

8. สรุปผลการปฏิบัติการและสิ่งที่ได้เรียนรู้จาก AI Agent

- สรุปผลการพัฒนา: สามารถพัฒนาโปรแกรม Tarot App สุ่มไพ่แบบ 3 ใบ (อดีต, ปัจจุบัน, อนาคต) ได้สำเร็จลุล่วงครบถ้วน
ไฟที่สุ่มได้ไม่ซ้ำกันในแต่ละรอบ พร้อมระบบเพลงประกอบ BGM ที่ปรับระดับเสียงได้ และมีระบบ Fallback ป้องกันโปรแกรม Crash
ในกรณีไฟล์ภาพสูญหาย
- การควบคุมและตรวจสอบ AI: การสร้างโค้ดด้วย AI สำหรับแอปที่มี Assets จำเป็นต้องกำชับเรื่องฟังก์ชัน resource_path เพื่อรองรับ
PyInstaller_MEIPASS เสมอ และต้องตรวจสอบโครงสร้างข้อมูล Dictionary ใน cards.py ให้ตรงกับชื่อไฟล์ภาพจริงทุกตัวอักษร
- ความรู้ที่ได้รับ: ได้เรียนรู้การแบ่งคอมโพเนนต์ GUI ด้วยคลาสลูก (SingleCardWidget), การใช้ random.sample
เพื่อรับประกันความไม่ซ้ำกันของข้อมูล, การเชื่อมต่อ Qt Multimedia ในโหมด Loop เพลงประกอบ และการบรรจุ Resource Assets
เข้าไปในตัวไฟล์ .exe